Course 2311: Analysis of Algorithms

Afsana Tasnim
Lecturer
Department of CSE
North Bengal International University

What is an Algorithm?

An algorithm is a sequence of unambiguous instructions for solving a problem i.e. for obtaining a required output for any legitimate input in a finite amount of time.

Notion of Algorithm

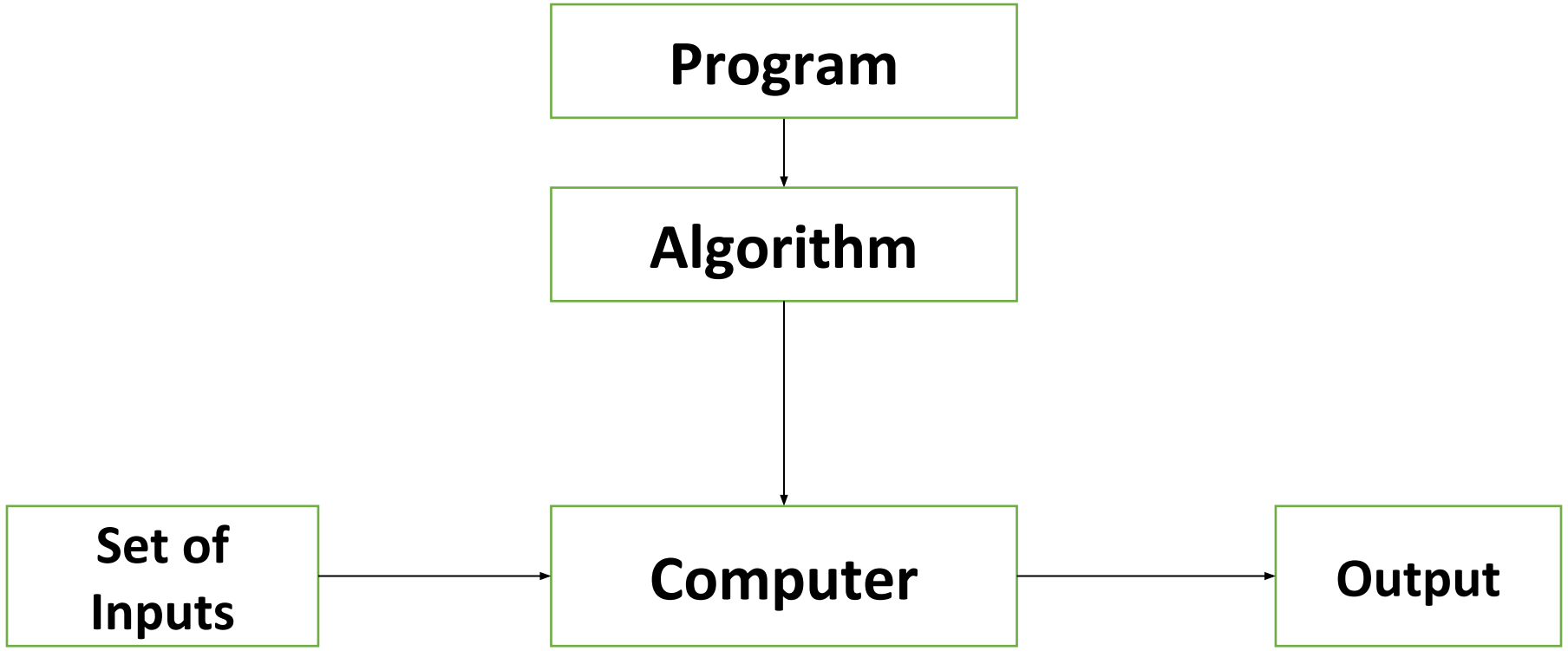


Fig 1: Notion of an Algorithm

Example: Sorting

Input: sequence $\langle a_1, a_2, \ldots, a_n \rangle$ of numbers.

Output: permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ such that $a'_1 \le a'_2 \le \cdots \le a'_n$.

Example:

Input: 8 2 4 9 3 6

Output: 2 3 4 6 8 9

Basic Issues Related to Algorithm

- How to design algorithms
- How to express algorithms
- Providing correctness of algorithms
- Efficiency
- Optimality

Algorithm Designing Strategies

- Brute Force
- Divide and Conquer
- Decrease and Conquer
- Transform and Conquer
- Greedy Approach
- Dynamic Programming
- Backtracking
- Space and time tradeoffs

Analysis of Algorithms

- How good is an algorithm?
    - Correctness
    - Time Efficiency
    - Space Efficiency
- Does there exist a better algorithm?
    - Lower Bounds
    - Optimality

What is an Algorithm

Recipe, process, method, technique, procedure, routine etc. with the following requirements:

1. Finiteness

   terminates after a finite number of steps

2. Definiteness

   rigorously and unambiguously specified

3. Input

   valid inputs are clearly specified

4. Output

   can be proved to produce the correct output for a valid input

5. Effectiveness

   steps are sufficiently simple and basic

Analysis of algorithms

Why study algorithms and performance?

- Algorithms help us to understand scalability.
- Performance often draws the line between what is feasible and what is impossible.
- Algorithmic mathematics provides a language for talking about program behavior.
- The lessons of program performance generalize to other computing resources.
- Framework for designing and analyzing algorithms for new problems.
- Speed is fun.

Some Well Known Computational Problems

- Sorting
- Searching
- Shortest paths in a graph
- Minimum Spanning Tree
- Travelling Salesman Problem
- Knapsack Problem
- Chess
- Tower of Hanoi

Bubble Sort

- Bubble sort is a simple sorting algorithm.

- This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order.

- This algorithm is not suitable for large data sets as its average and worst case complexity are of $O(n^2)$ where **n** is the number of items.

Bubble Sort

First pass

54	26	93	17	77	31	44	55	20	Exchange
26	54	93	17	77	31	44	55	20	No Exchange
26	54	93	17	77	31	44	55	20	Exchange
26	54	17	93	77	31	44	55	20	Exchange
26	54	17	77	93	31	44	55	20	Exchange
26	54	17	77	31	93	44	55	20	Exchange
26	54	17	77	31	44	93	55	20	Exchange
26	54	17	77	31	44	55	93	20	Exchange
26	54	17	77	31	44	55	20	93	93 in place after first pass

Bubble Sort

Algorithm for bubble sort:

```
begin BubbleSort(list)
    for all elements of the list
        if list[i] > list[i+1]
            swap(list[i], list[i+1])
        end if
    end for
    return list
end BubbleSort
```

Bubble Sort

- Pseudocode for bubble sort algorithm:

```
n= length[array]
counter= 1
while (counter<n-1)
    for i=0 to n-counter
        if array[i]>array[i+1]
            swap array[i] with array[i+1]
    counter++
```

Selection Sort

- The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from unsorted part and putting it at the beginning.

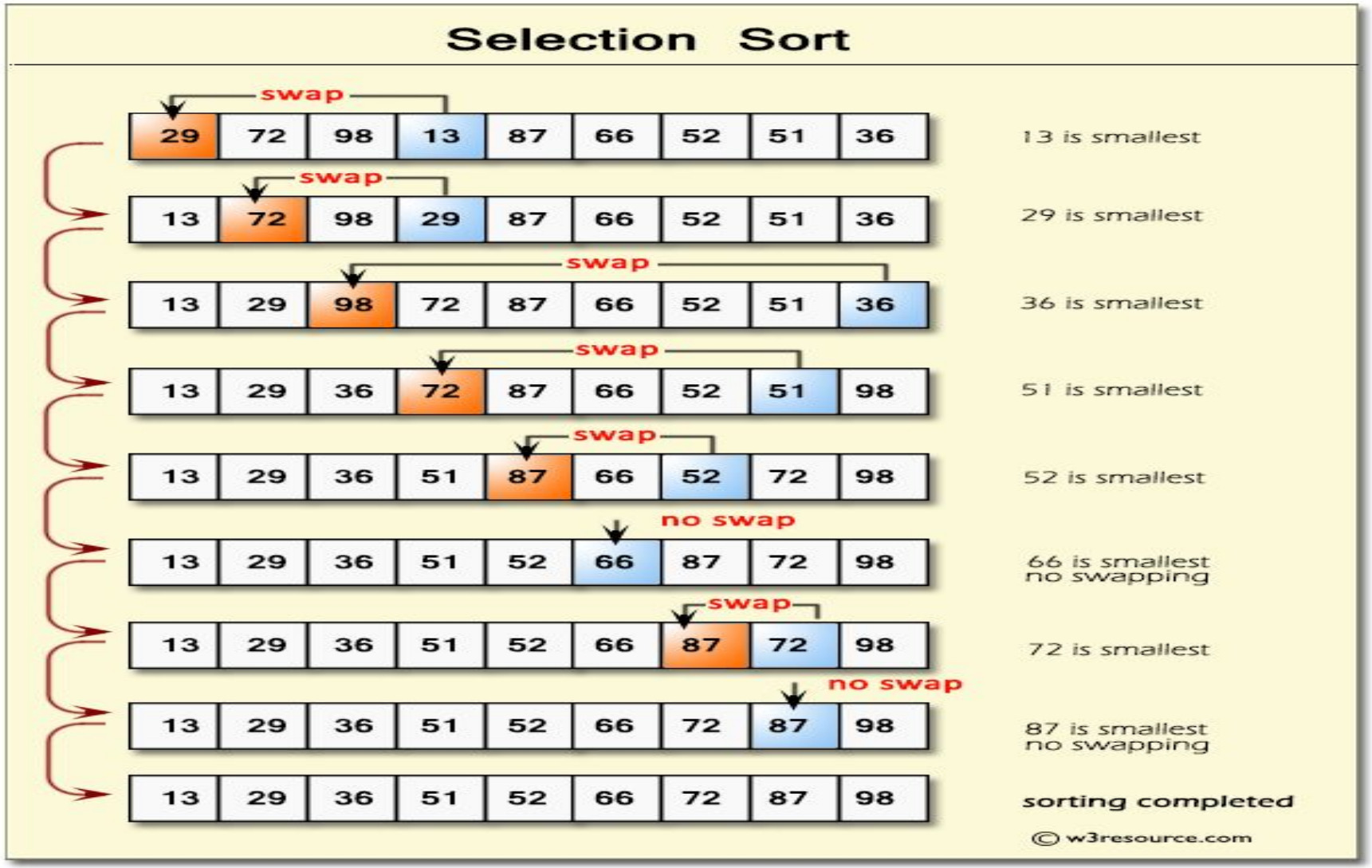
- The algorithm maintains two subarrays in a given array.

  1) The subarray which is already sorted.

  2) Remaining subarray which is unsorted.

- In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray.

Selection Sort



Selection Sort

The algorithm for selection sort:

```
SELECTION-SORT(A)
  n ← length[A]
  for j ← 1 to n - 1
      do smallest ← j
          for i ← j + 1 to n
              do if A[i] < A[smallest]
                     then smallest ← i
          exchange A[j] ↔ A[smallest]
```

8	4	6	9	2	3	1

Selection Sort

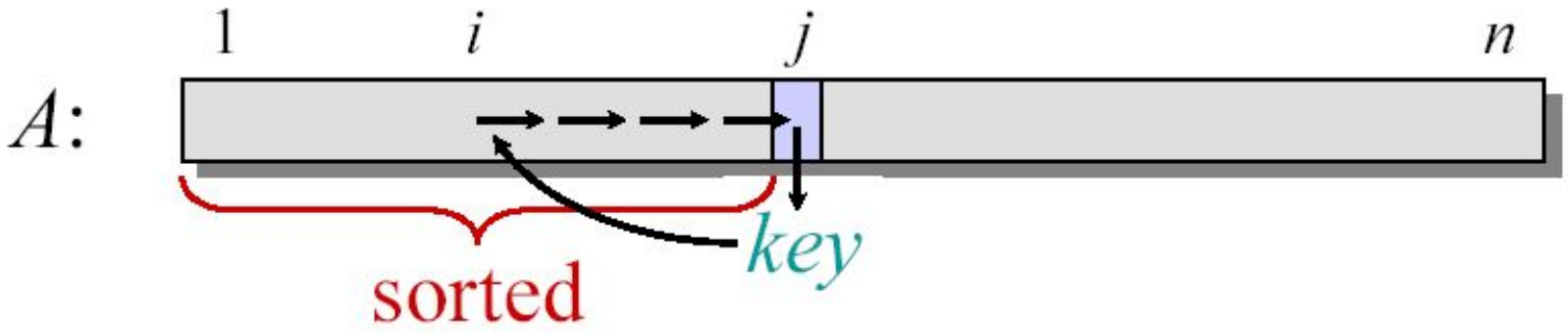
The pseudocode for selection sort:

```
n= length[array]
for i=0 to n-1
    for j=i+1 to n
        if array[j]<array[i]
            swap array[j] with array[i]
```

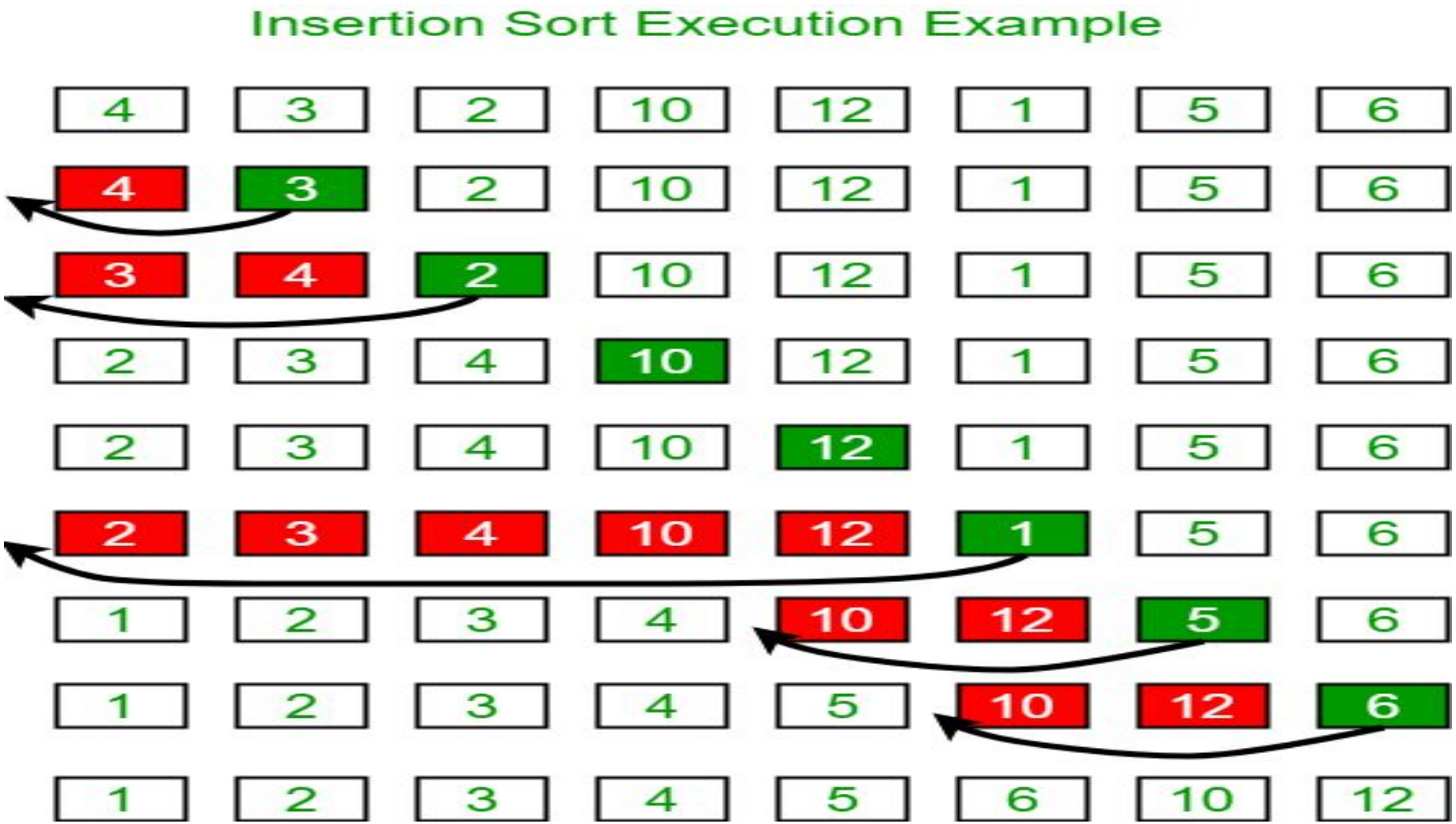
Insertion Sort

"pseudocode"

```
INSERTION-SORT (A, n)      ▷ A[1 . . n]
    for j ← 2 to n
        do key ← A[j]
           i ← j – 1
           while i > 0 and A[i] > key
               do A[i+1] ← A[i]
                  i ← i – 1
           A[i+1] = key
```



Insertion Sort



Running Time

- The running time depends on the input: an already sorted sequence is easier to sort.

- Parameterize the running time by the size of the input, since short sequences are easier to sort than long ones.

- Generally, we seek upper bounds on the running time, because everybody likes a guarantee.

Kinds of Analyses

Worst-case: (usually)

- T(n) = maximum time of algorithm on any input of size n.

Average-case: (sometimes)

- T(n) = expected time of algorithm over all inputs of size n.
- Need assumption of statistical distribution of inputs.

Best-case:

- Cheat with a slow algorithm that works fast on some input.

Machine-Independent time

The RAM Model

Machine independent algorithm design depends on a hypothetical computer called Random Acces Machine (RAM).

Assumptions:

- Each simple operation such as +, -, if ...etc takes exactly one time step.
- Loops and subroutines are not considered simple operations.
- Each memory acces takes exactly one time step.

Machine-independent time

What is insertion sort's worst-case time?

- It depends on the speed of our computer.
- Relative speed (on the same machine).
- Absolute speed (on different machines).

Machine-independent time: An example

A *pseudocode* for insertion sort (INSERTION SORT).

```
INSERTION-SORT(A)
for j ← 2 to length [A]
      do key ← A[ j]
      ▽ Insert A[j] into the sortted sequence A[1,..., j-1].
      i ←  j – 1
      while i > 0 and A[i] > key
            do A[i+1] ← A[i]
                  i ← i – 1
      A[i +1] ←  key
```

Analysis of INSERTION-SORT

INSERTION - SORT(A)	cost	times
for $j \leftarrow 2$ **to** $length[A]$	c_1	n
do $key \leftarrow A[j]$	c_2	$n-1$
∇ Insert $A[j]$ into the sorted sequence $A[1 \cdot\cdot j-1]$	0	$n-1$
$i \leftarrow j-1$	c_4	$n-1$
while $i > 0$ *and* $A[i] > key$	c_5	$\sum_{j=2}^{n} t_j$
do $A[i+1] \leftarrow A[i]$	c_6	$\sum_{j=2}^{n} (t_j - 1)$
$i \leftarrow i-1$	c_7	$\sum_{j=2}^{n} (t_j - 1)$
$A[i+1] \leftarrow key$	c_8	$n-1$

Analysis of INSERTION-SORT

The total running time is

$$T(n) = c_1 + c_2(n-1) + c_4(n-1) + c_5\sum_{j=2}^{n} t_j + c_6\sum_{j=2}^{n}(t_j - 1)$$
$$+ c_7\sum_{j=2}^{n}(t_j - 1) + c_8(n-1).$$

Analysis of INSERTION-SORT

The best case: The array is already sorted.

(t_j =1 for j=2,3, ...,n)

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1)$$
$$= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8).$$

$T(n) = an + b$

Here a and b both are constants that depend on the statement cost c_i . It is thus a linear function of n.

Analysis of INSERTION-SORT(contd.)

- The worst case: The array is reverse sorted

  (t_j =j for j=2,3, ...,n).

$$\sum_{j=1}^{n} j = \frac{n(n+1)}{2}$$

$$T(n) = c_1(n) + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1)$$

$$= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} + c_8\right)n - (c_2 + c_4 + c_5 + c_8)$$

$$T(n) = an^2 + bn + c$$

Asymptotic Analysis

- Asymptotic analysis of an algorithm refers to defining the mathematical boundation/framing of its run-time performance.

- We can very well conclude the best case, average case and worst case scenario of an algorithm by using the analysis.

- Asymptotic analysis is input bound. Other than the input all other factors are considered constant.

- Example:

Suppose the running time of one operation is computed as f(n) and may be for another operation it is computed as g(n^2). This means for the first operation the running time will increase linearly with the increase of n and for the second operation the running time will increase exponentially with increase of n. The running time for both the operations will nearly be the same if n is significantly small.

Asymptotic Notation

- Following are the commonly used asymptotic notations for calculating the running time complexity of an algorithm

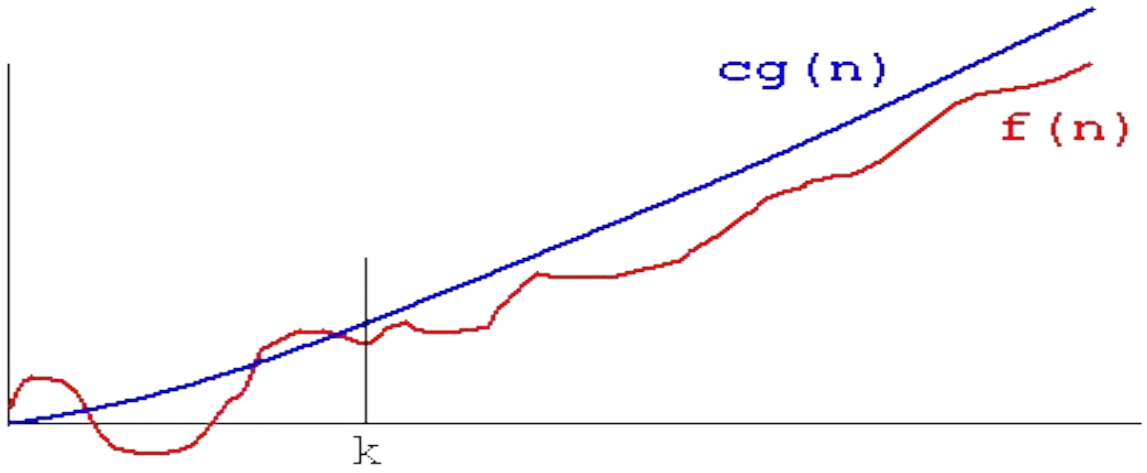
- O – Big Oh Notation f(n) $\leq c.g(n); c > 0, n \geq$ k, k ≥ 0

- Ω – Omega Notation f(n) $\geq c.g(n)$;

- Θ – Theta Notation $c_1 g(n) \leq$ f(n) $\leq c_2 g(n)$;

Big Oh Notation (O)

- Big-O notation represents the upper bound of the running time of an algorithm. Thus, it gives the worst-case complexity of an algorithm.



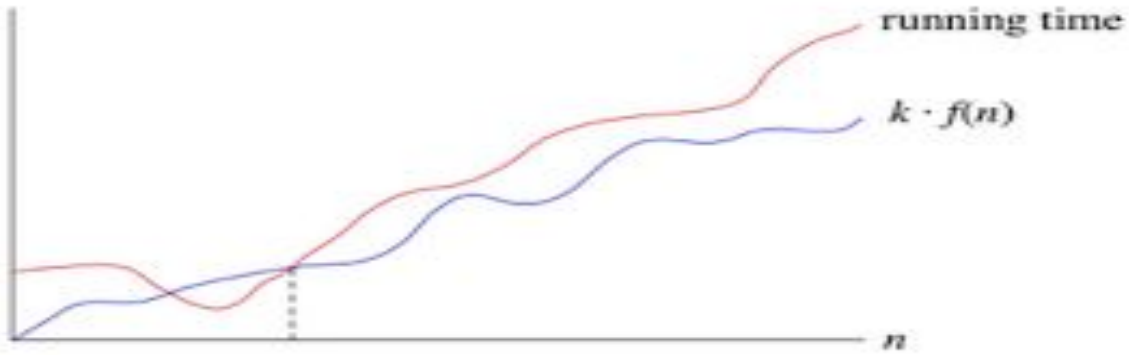
O(g(n)) = { f(n): there exists positive constants c and $n_0$such that 0≤f(n)≤C(g(n)) for all n≥ n_0}

The above expressing can be described as a function f(n) belons to the set O(g(n)) if there exists a positive constant c such that it lies between 0 and cg(n) for sufficiently large n.

Omega Notation (Ω Notation)

Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.

If a running time is $\Omega(f(n))$, then for large enough n, the running time is at least $k \cdot f(n)$ for some constant k. Here's how to think of a running time that is $\Omega(f(n))$:

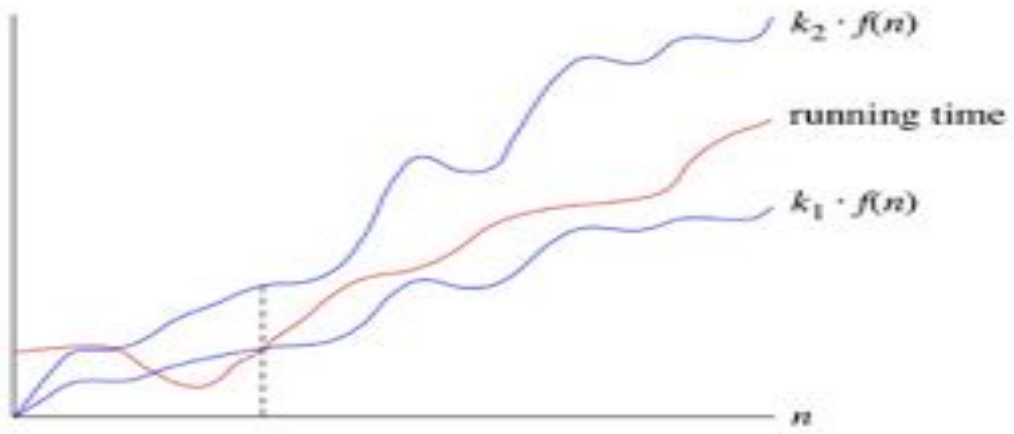


We say that the running time is "big-Ω of $f(n)$." We use big-Ω notation for **asymptotic lower bounds**, since it bounds the growth of the running time from below for large enough input sizes.

Theta Notation (θ Notation)

Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.

We are not restricted to just n in big-Θ notation. We can use any function, such as n^2, $n \lg n$, or any other function of n. Here's how to think of a running time that is $\Theta(f(n))$ for some function $f(n)$:



Once n gets large enough, the running time is between $k_1 \cdot f(n)$ and $k_2 \cdot f(n)$.